

FRED TALKS

17th August 2026

CONTENTS

Headless everything for personal AI

INTERCONNECTED · 1335 WORDS

Agentic search - retrieval, harness, or model?

DOUG TURNBULL · 1535 WORDS

Big tech engineers need big egos

SEANGOEDECKE.COM RSS FEED · 1975 WORDS

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 1776 WORDS

Headless everything for personal AI

INTERCONNECTED · 18 APR 2026 · [SOURCE](#)

It's pretty clear that apps and services are all going to have to go *headless*: that is, they will have to provide access and tools for personal AI agents without any of the visual UI that us humans use today.

By services I mean things like: getting a new passport; finding and booking a hotel or a flight; managing your bank account; shopping for t-shirts with a minimum cotton weight from brands similar to ones that you've bought from before.

Why? Because using personal AIs is a better experience for users than using services directly (honestly); and headless services are quicker and more dependable for the personal AIs than having them click round a GUI with a bot-controlled mouse.

Where this leaves design for the services... well, I have some thoughts on that.

Headless services? They're happening already.

Already there is [MCP, as previously discussed](#) (2025), a kind of web API specifically for AI. For instance, best-in-class call transcriber app Granola [recently released their MCP](#) and now you can ask your Claude to pull out the meeting actions and go trawling in your personal docs to find answers to all the question. A good integration.

But command-line tools are growing in popularity, although they used to be just for developers. So you can now create a spreadsheet by typing in your terminal:

```
gws sheets spreadsheets create --json '{"properties": {"title":  
"Q1 Budget"}}'
```

Here are some recently launched CLI tools:

- [gws](#): Google Workspace CLI, Drive, Gmail, Calendar, and every Workspace API. Zero boilerplate. Structured JSON output. 40+ agent skills included.
- [Obsidian CLI](#) to do anything you can do with the (very popular) note-taking app - like keep daily notes, track and cross off tasks, search - from the CLI

- Salesforce CLI and, look, I don't really understand what the Salesforce business operating system does, but the fact it has a CLI too is significant.

And here is CLI-Anything (31k stars on GitHub) which auto-generates CLIs for ANY codebase.

Why CLIs?

It turns out that the best place for personal AIs to run is on a computer. Maybe a virtual computer in the cloud, but ideally your computer. That way they can see the docs that you can see, and use the tools that you can use, and so what they want is not APIs (which connect webserver) but little apps they can use directly. CLI tools are the perfect little apps.

CLIs are composable so they are a better fit for what users actually do.

By composable I mean you can: query your notes then jump to a spreadsheet then research the web then jump back to the spreadsheet then text the user a clarifying question then double-check your notes, all by bouncing between CLIs in the same session.

A while back app design got obsessed with “user journeys.” Like the journey of a user finding a hotel then booking a hotel then staying in it and leaving a review.

But users don't live in “journeys.” They multitask; they talk to people and come back to things; they're idiosyncratic Try grabbing the search results from the Airbnb app and popping them in a message to chat on the family WhatsApp, then coming back to it two days later. It's a pain and you have to use screenshots because apps and their user journeys are not composable.

CLIs are composable because they came originally from Unix and that is the Unix tools philosophy: tools were designed so that they could operate together.

Personal AIs like Openclaw or [Poke](<https://poke.com>) do what users want and don't follow “designed” user journeys, and as a result the composed experience is more personal and way better. CLIs are a great underlying technology for that.

CLIs are smaller than regular apps and so they are easier to secure.

The alternative to providing special tools for AIs is that the AIs use the same browser-based apps that we do, and that's a terrifying prospect.

For one, AIs are really good at finding security holes. The new Mythos model from Anthropic is so good at discovering security flaws that it has been held back from the public and governments are convening emergency meetings of the biggest banks.

And including a UI increasing complexity and makes security holes more likely.

Here's a recent shocking example. Companies House is the national register of all companies, directors, and accounts for England and Wales. Users could view *and edit* any other user's account:

a logged-in company director could exploit the flaw by starting from their own dashboard and then trying to log into another company's account.

Once they reach the 2FA block, which they would not be able to pass, all that was required was to click the browser's back button a few times. Typically, the user would be taken back to their own dashboard, but the bug instead returned them to the company they had tried to log into but couldn't.

This bug had been present since October 2025.

Imagine a future where personal AIs are filing company records, and one of them notices this security hole overnight and posts about it on moltbook or whatever agent-only social network is most popular. The other agents would exploit the system up, down and sideways before the engineering team woke up.

The only viable solution is that services need to security hardened, and to do that they need to be simplifying and minimised. Again, CLI tools are a great fit.

What does this mean for front-end design?

Design won't go anywhere.

Sure, the front-end should be driving the same CLI tools that agents use.

Arguably it's more important than ever: human users will encounter services, figure out what they can do, and pick up their vibe from using the app, just as now.

Then they'll tell their personal AI about the service and never see the front-end again, or re-mix it into bespoke personal software.

So from a usability perspective I see front-end as somewhat sacrificial. AI agents will drive straight through it; users will encounter it only once or twice; it will be customised or personalised; all that work on optimising user journeys doesn't matter any more.

But from a vibe perspective, services are not fungible. e.g. if you're finding a restaurant then Yelp, Google Maps, Resy, and The Infatuation are all more-or-less equivalent for answering that question but clearly they are completely different and you'll use different services at different times.

Understanding that a service is *for you* is 50% an unconscious process - we call it brand - and I look forward to front-end design for apps and services optimising for brand rather than ease of use.

If I were a bank, I would be releasing a hardened CLI tool like *yesterday*.

There is so much to figure out:

- How do permissions work? Should the user get a notification from their phone app when the agent strays outside normal behaviour? How do I give it credentials to act on my behalf, and how long do they last?
- How does adjacency work? My bank gives me a current account in exchange for putting a "hey, get a loan!" button on the app home screen. How do you make offers to an agent?

Headless banks.

Headless government?

I'd love to show you a worked example here. I vibed up a suite of four CLI tools that wrap four different services from UK government departments.

If I were renting a house, I would set my agent to learning about the neighbourhoods using one of these tools. Another tool will be helpful next time I'm buying a used car. (There's a Companies House command-line tool too.)

But I won't show you the tools because I don't want to be responsible for maintaining and supporting them.

I wish Monzo came with an official CLI. I wish Booking.com came with a CLI. I reckon, give it a year, they will.

Auto-calculated kinda related posts:

If you enjoyed this post, please consider sharing it by email or on social media. [Here's the link.](#)
Thanks, —Matt.

Agentic search - retrieval, harness, or model?

DOUG TURNBULL · 09 JUN 2026 · [SOURCE](#)

Agentic search gets interesting when agents do not know how to find the right answer.

Oh, the agent might think it knows. It might confidently BS us. But the agent's poor domain intuition steers itself astray.

Agents make false assumptions about what *our* users think is relevant. Our fashionista users think "red shoes" should return high-heels. When I worked at one company ABE wasn't a president, it was an A/B testing tool. Agents need context to know these things - and [context engineering needs agentic search](#)

Confusingly, depending who you talk to, agentic search can mean one of three distinct implementation patterns:

- **Retrieval-centric** - just build good search so agents can use it to fill in missing context
- **Harness-centric** - steer the agent towards needed context, even with bad search
- **Model-centric** - fine tune an LLM to know how to search *our* data

In this post, I'll walk through each. With a bit more breadth, you might appreciate which flavor your colleagues seem to mean when they say "agentic search"

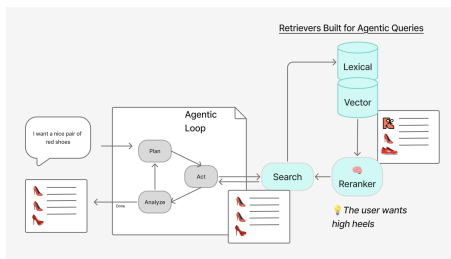
Retrieval-Centric Implementation

When frontier models don't know, they search. Ask ChatGPT a question about news, it'll search. Ask it about a very specific technical problem, it'll search. During training, LLMs see *search* examples as a technique to learn what it doesn't know. To find what it needs.

Therefore, we just need to build good search to solve any conceivable query from an agent.

Let's assume we run an e-commerce catalog. For us, when users search "red shoes" they mean "red high heels". But the agent doesn't know that. Luckily it asks *search*. Below we see it returns the right results.

Below we see initial retrieval to lexical / vector backends pulls back some reasonable, if naive, “red shoe” candidates. Still that’s not quite right. Luckily the reranker shapes the results towards our understanding of that intent.



Of course we might have other components here - query understanding, diversity, custom embedding models, and more.

The important point is that *search leads the agent by the nose* towards what’s relevant. We assume search can define what a good “red shoe” is, overriding the agent’s perspective.

Most teams build retrieval-centric approaches with classic RAG. Chunks of answers, and embeddings trained to recognize them as answers.

Unfortunately, answers don’t always look tied to the question. For example:

Question:

Synopsis of the book Ubik

Answer:

By the year 1992, humanity has colonized the Moon and psychic powers are common. The protagonist, Joe Chip, is a debt-ridden technician working for Runciter Associates, a “prudence organization” employing “inertials”—people with the ability to negate the powers of telepaths and “precogs”—to enforce the privacy of clients.

If you don’t know the book Ubik, it’s not clear that this answers the question. The agent says “cool story bro” and ignores the info.

Search like this actually is divorced from the web search trusted by frontier models. The web contains titles, headings, and other elements placing the answer in context.

The big downside? Search remains hard. We're not Google.

Most importantly, we don't have perfect search like Google or Bing. Even with good search - almost nobody builds *google quality results*. And since agents trust search - simple distractors in retrieval can, as Lester Solbakken says, easily confuse reasoning.

Harness-centric implementations

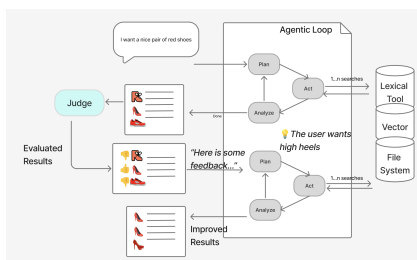
Who should be in charge? Should the agent manage the search process? Or should search drive the agent?

Moving the emphasis to the harness, we put the agent in charge.

Imagine stripping search tools down to core retrieval primitives. Just a BM25 backend. Or a filesystem with CLI tools. We tell the agent find to what it needs with these untuned tools. The agent might struggle more, but hey, its smart, it can get figure it out. Right?

Still, the agent might find what it thinks is relevant, as it does in the image below. Sadly that's not what's actually relevant. To help the agent, we inject external knowledge. We let a judge direct the agent, correcting its mistakes, and guiding it towards better search strategies.

So when the agent returns results, it's not the "user" that receives the candidates. Instead a judge labels results as relevant or not. The agent gets the hint, finding results similar to those labeled relevant. Avoiding those labeled irrelevant. To steel from Jo Kristian Bergem, this is relevance feedback on steroids.



This works. Look at the ESCI dataset. If I have an oracle labeling with judgments from these datasets, I get quite significant improvements.

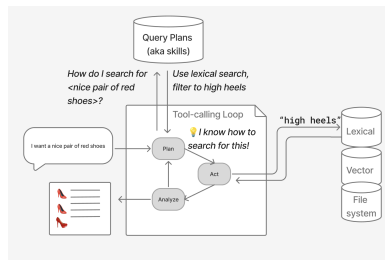
Variant	NDCG@10	Description
ESCI BM25	0.2895	Simple BM25 weighing name / description
ESCI agentic	0.4101	GPT5-mini tool calling loop w/ BM25+e5 embeddings tools

ESCI agentic w/
oracle 0.5843

GPT5-mini tool calling loop w/ BM25+e5 embeddings tools.
Judge responds with feedback once

Here the second row we trust the agent's interpretation of queries. That provides a large gain over BM25. But inject more domain knowledge via a judge (the third row) and agent performance improves dramatically.

We can guide agents with information ahead of time too. For example, I've seen teams take inspiration from skills in coding agents. Imagine a targeted query plan that tells the agent how to use it's tools to solve a specific problem.



Optimizing content helps harnesses

We help agents even further by optimizing content for findability. That's the trick we've played on ourselves with coding agents. We write docs to explain to agents why something's useful. We don't just rely on that dumb chunk, we document the purpose of the knowledge:

```
/books/scifi/ubik.txt
```

```
## About Book Ubik Ubik is a book by Philip K. Dick
```

```
## Synopsis
```

```
By the year 1992, humanity has colonized the Moon and psychic powers are common. The protagonist, Joe Chip, is a debt-ridden technician working for Runciter Associates, a "prudence organization" employing "inertials"—people with the ability to negate the powers of telepaths and "precogs"—to enforce the privacy of clients.
```

Unlike a naive chunk, this bit of information has purpose. It's clear, when its in the context, what problem it solves.

By organizing + cataloging information we've done what every search developer wishes their content team would do: actually optimize content to be findable.

The downside: cost

The major downside here? Token costs. The exploration here requires more than just issuing a search and retrieving one set of results. It's an agent's exploration of the environment. And that exploration starts fresh with every new context window.

Model-centric approach

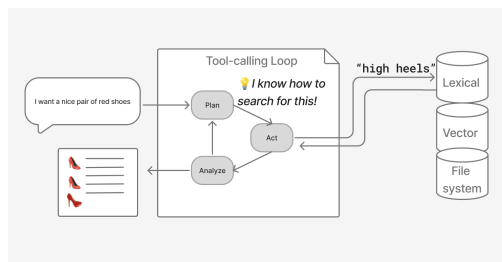
What if we literally fine-tuned the LLM to search efficiently?

The harness based approach takes a long, winding path towards relevant results. With every fresh context, it wastes calls relearning what works over-and-over. It's costly and slow.

Could we train a model that doesn't repeat the same mistakes with our tools?

Our judge helps label some of the agent's reasoning / tool-calling paths as useful. And label others less so. Once you have labeled traces, you can fine-tune an open weights model to proactively, efficiently make the right tool calls given the input.

Below, for example, the model knows exactly how to search for our "red shoes" query given its extensive fine tuning.



Now we've built a model targeting the search task. We don't tune our retrievers much. Instead we continue to use the simpler / dumb search tools.

Even cooler, the agentic search model's behavior can change just by adjusting the system prompt. "This is an e-commerce search task, optimize for top 10 ranking" might be different from "You're retrieving chunks for financial research, maximize diversity and relevance at the same time."

Further the models can stay smaller. By just focusing on the search part - not the entire domain problem - they can shrink to a manageable size allowing them to be self hosted.

The downside? Early days

The OG in the space is SID. Other contenders like Glean's Waldo model have sense entered. They're all promising, but not widely adopted. Many unknown unknowns exist. In a sense, we don't yet know the upside or downsides yet. I suspect though this paradigm has tremendous promise in the years ahead.

In other words - stay tuned!

The alchemy of ambiguity

People use "agentic search" confusingly and inconsistently. They focus on their part of the problem. That means we have a collection of solutions like enriched base metals - and its unclear what alchemy brings them together.

We're converging on a fascinating mix of easier to organize data (like PageIndex). I'm seeing patterns in harness design that mirror coding agents - hooks / evaluators that steer agents. Or "skills" that guide them a priori. Much like with coding - the agents eat the harnesses. When a successful pattern emerges, agents, in our case agentic search models, train to memorize these patterns.

Retrieval also seems to be moving towards agent-centric approaches. Agents have their own specific, unique retrieval patterns. We're finally seeing late interaction + learned sparse retrieval approaches have their day from companies like LightOn and MixedBread

We're swimming in an interesting retrieval primordial goop, what comes out may be an obvious combination - or look nothing like the component parts. Exciting times!

Enjoy softwareDoug in training course form!

Starting May 18!

Signup here - <http://maven.com/softwareDoug/cheat-at-search>



I hope you join me at [Cheat at Search with Agents](#) to learn use agents in search, build better RAG and use LLMs in query understanding.

Big tech engineers need big egos

SEANGOEDECKE.COM RSS FEED · 14 MAR 2026 · [SOURCE](#)

It's a [common position](#) among software engineers that big egos have no place in tech¹. This is understandable - we've all worked with some insufferably overconfident engineers who needed their egos checked - but I don't think it's correct. In fact, I don't know if it's possible to survive as a software engineer in a large tech company without some kind of big ego.

However, it's more complicated than "big egos make good engineers". The most effective engineers I've worked with are simultaneously high-ego in some situations and surprisingly low-ego in others. What's going on there?

Engineers need ego to work in large codebases

Software engineering is shockingly humbling, even for experienced engineers. There's a reason this joke is so popular:



The minute-to-minute experience of working as a software engineer is dominated by *not knowing things* and *getting things wrong*. Every time you sit down and write a piece of code, it will have several things wrong with it: some silly things, like missing semicolons, and often some major things, like bugs in the core logic. We spend most of our time fixing our own stupid mistakes.

On top of that, even when we've been working on a system for years, we still don't know that much about it. I wrote about this at length in *Nobody knows how large software products work*, but the reason is that big codebases are just that complicated. You simply can't confidently answer questions about them without going and doing some research, even if you're the one who wrote the code.

When you have to build something new or fix a tricky problem, it can often feel straight-up impossible to begin, because good software engineers know just how ignorant they are and just how complex the system is. You just have to throw yourself into the blank sea of millions of lines of code and start wildly casting around to try and get your bearings.

Software engineers need the kind of ego that can stand up to this environment. In particular, they need to have a firm belief that they *can* figure it out, no matter how opaque the problem seems; that if they just keep trying, they can break through to the pleasant (though always temporary) state of affairs where they understand the system and can see at a glance how bugs can be fixed and new features added².

Engineers need ego to work in big tech companies

What about the non-technical aspects of the job? Nobody likes working with a big ego, right? Wrong. Every great software engineer I've worked with in big tech companies has had a big ego - though as I'll say below, in some ways these engineers were surprisingly low-ego.

You need a big ego to take positions. Engineers love being non-committal about technical questions, because they're so hard to answer and there's often a plausible case for either side. However, as I keep saying, engineers have a duty to take clear positions on unclear technical topics, because the alternative is a non-technical decision maker (who knows even less) just taking their best guess. It's scary to make an educated guess! You know exactly all the reasons you might be wrong. But you have to do it anyway, and ego helps *a lot* with that.

You need a big ego to be willing to make enemies. Getting things done in a large organization means making some people angry. Of course, if you're making lots of people angry, you're probably screwing up: being too confrontational or making obviously bad decisions. But if you're making a large change and one or two people are angry, that's just life. In big tech companies, any big technical decision will affect a few hundred engineers, and one of them is bound to be unhappy about it. You can't be so conflict-averse that you let that stop you from doing it, if you believe it's the right decision. In other words, you have to have the confidence to believe that you're right and they're wrong, even though technical decisions always involve unclear tradeoffs and it's impossible to get absolute certainty.

You need a big ego to correct incorrect or unclear claims. When I was still in the philosophy world, the Australian logician Graham Priest had a reputation for putting his hand up and stopping presentations when he didn't understand something that was said, and only allowing the seminar to continue when he felt like he understood. From his perspective, this wasn't rude: after all, if *he* couldn't understand it, the rest of the audience probably couldn't either, and so he was doing them a favor by forcing a more clear explanation from the speaker.

This is obviously a sign of a big ego. It's also a trait that you need in a large tech company. People often nod and smile their way past incorrect technical claims, even when they suspect they might be wrong - assuming that they've just misunderstood and that somebody else will correct it, if it's truly wrong. **If you are the most senior engineer in the room, correcting these claims is your job.**

If everyone in the room is so pro-social and low-ego that they go along to get along, decisions will get made based on flatly incorrect technical assumptions, projects will get funded that are impossible to complete, and engineers will burn weeks or months of their careers vainly trying to make these projects work. You have to have a big enough ego to think "actually, I think I'm right and everyone in this room is confused", even when the room is full of directors and VPs.

Sometimes you need to put your ego aside

All of this selects for some pretty high-ego engineers. But in order to actually *succeed* in these roles in large tech companies, you need to have a surprisingly low ego at times. **I think this is why really effective big tech engineers are so rare: because it requires such a delicate balance between confidence and diffidence.**

To be an effective engineer, you need to have a towering confidence in your own ability to solve problems and make decisions, even when people disagree. But you also need to be willing to instantly subordinate your ego to the organization, when it asks you to. At the end of the day, your job - the reason the company pays you - is to execute on your boss's and your boss's boss's plans, whether you agree with them or not.

Competent software engineers are allowed quite a lot of leeway about *how* to implement those plans. However, they're allowed almost no leeway at all about the plans themselves. In my experience, being confused about this is a common cause of burnout³. Many software engineers are used to making bold decisions on technical topics and being rewarded for it. Those software engineers then make a bold decision that disagrees with the VP of their organization, get immediately and brutally punished for it, and are confused and hurt.

In fact, **sometimes you just get punished and there's nothing you can do.** This is an unfortunate fact of how large organizations function: even if you do great technical work and build something really useful, you can fall afoul of a political battle fought three levels above your head, and come away with a *worse* reputation for it. Nothing to be done! This can be a hard pill to swallow for the high-ego engineers that tend to lead really useful technical projects.

You also have to be okay with having your projects cancelled at the last minute. It's a very common experience in large tech companies that you're asked to deliver something quickly, you buckle down and get it done, and then right before shipping you're told "actually, let's cancel that, we decided not to do it". This is partly because the decision-making process can be pretty fluid, and partly because many of these asks originate from off-hand comments: the CTO implies that something might be nice in a meeting, the VPs and directors hustle to get it done quickly, and then in the next meeting it becomes clear that the CTO doesn't actually care, so the project is unceremoniously cancelled⁴.

Final thoughts

Nobody likes to work with a bully, or with someone who refuses to admit when they're wrong, or with somebody incapable of empathy. But you really do need a strong ego to be an effective software engineer, because software engineering requires you to spend most of your day in a position of uncertainty or confusion. If your ego isn't strong enough to stand up to that - if you don't believe you're good enough to power through - you simply can't do the job.

This is particularly true when it comes to working in a large software company. Many of the tasks you're required to do (particularly if you're a senior or staff engineer) require a healthy ego. However, there's a kind of catch-22 here. If it insults your pride to work on silly projects, or to occasionally "catch a stray bullet" in the organization's political fights, or to have to shelve a project that you worked hard on and is ready to ship, you're too high-ego to be an effective software engineer. But if you can't take firm positions, or if you're too afraid to make enemies, or you're unwilling to speak up and correct people, you're too low-ego.

Engineers who are low-ego in general can't get stuff done, while engineers who are high-ego in general get slapped down by the executives who wield real organizational power. The most successful kind of software engineer is therefore a chameleon: low-ego when dealing with executives, but high-ego when dealing with the rest of the organization⁵.

1. What do I mean by "ego", in this context? More or less the colloquial sense of the term: a somewhat irrational self-confidence, a tendency to believe that you're very important, the sense that you're the "main character", that sort of thing

↩

2. Why is this "ego", and not just normal confidence? Well, because of just how murky and baffling software problems feel when you start working on them. You really do need a degree of confidence in yourself that feels unreasonable from the inside. It should be obvious, but I want to explicitly note that you don't *just* need ego: you also have to be technically strong enough to actually succeed when your ego powers you through the initial period of self-doubt.

↩

3. I share the increasingly-common view that burnout is not caused by working too hard, but by hard work unrewarded. That explains why nothing burns you out as hard as being punished for hard work that you expected a reward for.

↩

4. It's more or less exactly this scene from Silicon Valley.

↩

5. This description sounds a bit sociopathic to me. But, on reflection, it's fairly unsurprising that competent sociopaths do well in large organizations. Whether that kind of behavior is worth emulating or worth avoiding is up to you, I suppose.

↩

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 08 JUL 2026 · [SOURCE](#)

You can also read this post on [Prime Radiant's blog](#).

Almost all of this post was written on June 1, before Anthropic shipped Claude Tag. Nothing I'm writing about uses (or is based on) Claude Tag

At [Prime Radiant](#), we've got a bunch of agents in our Slack. I want to tell you a bit about how we've started shipping some changes to internal production with an "agentic user in the loop." Letting an agent work directly with the implementer building its runloop and tooling is getting us higher-quality tools and experiences for the agent without a human in the middle of a game of telephone.

But first, a bit of backstory on how we're using non-coding agents at Prime Radiant.

One of them, Scribble, is just there to listen for when someone reports a problem or provides a new bit of information. It opens tickets for the issues and updates an internal wiki with the information. (It also logs our daily standups and keeps us honest about whether we finished what we said we were going to do the previous day.)

Another, Nora, is our junior go-to-market "person." Nora is built on top of Nanoclave and is still learning her craft, but is surprisingly helpful, partially because she considers herself a team member, not just an assistant. I've put a bunch of time into tuning her prompting and constitution. She gets a ton of value out of knowing that she journals obsessively. (I should note that I don't tend to gender my agents, but Nora picked a name and a gender and was very clear about that and...who am I to argue?)

I think the most surprising interaction I've had with Nora was the morning I woke up to a slack DM telling me that I needed to be speaking at more conferences. (I do.) She pointed out that the CFP for a particular AI conference had closed a week earlier, but that the program chair was known to take late submissions of interesting talks by DM. She'd taken the liberty of drafting a DM for me to send. I didn't actually *send* the DM, but I should have.

We have another agent, "Spec-together" that was a first prototype of what has become our new multi-player Brainstorm app.

And we have a small fleet of "Sen" personal assistant agents. They're **claw-esque*, but date from before openclaw was a thing. Each one has been trained to be an executive assistant, using a set of skills built on some of the resources that excellent human EAs use to learn their craft. Sen triages my mail, puts together a daily brief, does research tasks (both for me and independently), interacts with Linear, etc.

They have the standard set of affordances - They can chat on Slack. They use tools. They've got a task scheduler that can wake them up and re-inject a prompt they set for themselves at a specific time or on a schedule they define. They can use and author skills. Mine has a dropbox folder it shares with me. When we moved Sen to the cloud, I lost one of my favorite dumb features - Sen used to be able to print things directly. Sen (v1) was built on top of the Claude Agents SDK.

For the past month or so, I've been doing initial bringup of a "new" iteration of Sen, intended to be more of a colleague than an assistant. This time around, I'm building it from the ground up on top of an agents SDK I control. Lace got its start in May 2025 as my take on a command line coding agent. Over the past year, it's mutated a few times, first growing a web interface, then flipping to an ACP-derived wire protocol so you can connect any client you want to it. It speaks a bunch of different flavors of model provider API, manages caching, subagents, tools, skills, and all the other stuff you might expect. It has supported subagents in isolated containers for the better part of a year. One of the latest things to land has been the ability to run subagents locally and to project *all* of their tools into a container, such that the agent believes it is running in that container.

The thing I've been working on over the past few weeks is the credentials story for the new Sen agent running on top of Lace. Right now, I'm building with a design that assumes the agent has its own credentials for any service that it accesses, not yours. (You wouldn't share your email account or GitHub credentials with a colleague, right?) As of now, nobody has solved Simon's Lethal Trifecta - If a single agent has access to private information, can communicate externally, and is exposed to untrusted content, there is no structural way to guarantee that that agent can't be suborned. So the name of the game is compartmentalization and risk *reduction*.

The architecture I've landed on for now:

- The main agent does not have the ability to directly communicate externally.
- The main agent is able to communicate with ephemeral subagents that *do* have the ability to communicate externally
- No agent has intentional direct exposure to credentials (with one very real gap that I haven't solved yet)

All credentials for the agent live in a 1Password vault. All outgoing https traffic goes through a onecli-style transparent MITM (man in the middle) proxy. When a subagent wants to do something that requires a credential, it runs a command that asks an *arbiter* agent running in another container for permission to use that credential. If the arbiter decides the request is reasonable, it provides the subagent with a random string they can use in place of the requested credential. When the agent uses that random string in an outbound request to the right remote host, the MITM proxy rewrites the random string into the real credential on the fly.

The case that's a little more complicated is when the agent wants to do something that requires a browser, like logging into Gmail as themselves or using a service that has no API. For a bunch of reasons, the MITM rewrite trick does not work as reliably in a browser context. Often the site/app's javascript plays games with the password to hash the password before sending it. Or does some other weird validation. Or is actually sending it to a different backend service. So far, the best I've come up with keeps the credential out of the agent's transcript and prevents unintentional disclosure. In these cases, the subagent running in the container generates a random temporary password, uses superpowers-chrome to fill it into the password field, and then runs a command that asks the arbiter agent (running in its own container) for help. The arbiter, if it decides the request is reasonable, instructs a helper tool to log into the subagent's container and replace the random password with the real one. It's not perfect and there's more we can do to lock things down, but it's a better first pass than anything I've had to date.

What actually got me writing today was not this architecture, but the *development pattern* I've been using to build this. On the one side, I've got a Claude Code session. I hooked it up to our Slack instance with a self-provisioned commandline slack client Drew built called Slackline - Slackline is designed to make it easy for agents that don't *live* on Slack to participate in discussions there. It has primitives for sending messages, reading chats, waiting for replies, etc. The whole command-line slackline tool has been built for agents as its primary user. Sure, a human *could* use it to use slack from the commandline. And sure, it has enough of a json api that you could use it as part of traditional automation. But it's *for* agents.

It took a bit of iteration to get to the point where the new Sen 2.0 agentic colleague harness was live on Slack. But once it was there, I could talk through its experience of its tools and memory and credentials and workspaces. And the issues showed up pretty quickly.

The "right" kind of context compaction for a coding agent is desperately wrong for a long-lived persona that might be engaging in multiple simultaneous conversations. And the only way to see if the credentials management tools work is to try to do something crazy, like sign into GitHub from a subagent session.

And sure, I could take this feedback into a Claude Code (or Codex) session and work it through. But I've got too many projects in flight and there's very little reason for me to be in the loop on most of this work. I would just slow it down. So I told Claude Code to use slackline to talk to "@Ada-sen" about the work it was doing.

Claude pinged Ada in #bot-testing to describe the work we'd just done and to ask Ada if it could test it out. Ada dutifully fired up a subagent and tried to log into GitHub. Claude watched the logs to see if the credential proxy did the right thing. It did not.

That kicked off a number of days of iteration. Claude would propose a change to Ada. Ada would review Claude's spec, raising questions or concerns. Claude would update the spec. Once Claude and Ada were in agreement, Claude would build an updated version of Ada, make sure Ada wasn't in the middle of something, and then deploy the update. Once Ada was up and running again, Claude would @Ada-sen, explain the test it wanted to run, and wait for results.

Once a bit of functionality works, Claude checks in with Ada to see what could make it simpler and easier to use. Ada typically kicks off some subagents to see whether they stumble through the new feature and then reports back.

At one point, I wrote to Claude "I've got to get to bed. When this project is done, check with Ada to see whatever quality of life features you could build." I woke up to a laundry list of about a dozen harness improvements. Overnight, Claude had solicited a wishlist from Ada. Ada ranked a bunch of requests. Claude ordered them and wrote out a spec for everything it was comfortable building. Ada reviewed the specs, then Claude built the features. Ada tested them out and requested changes. Claude updated the features until Ada was happy. Once Ada signed off, Claude merged the changes to main and wrote up an after-action report for me.

It's been pretty magical to watch.